**Table of Headings (as per 20-mark structure)**


1. Introduction to Mobile Computing & Domain
2. Comparison of Mobile Platforms (Android, iOS, etc.)
3. Revision of Java for Android (OOP recap)
4. Android Platform & Dalvik VM / ART
5. Android Building Blocks (Activity, Service, Content Provider, Broadcast Receiver)
6. Activity Lifecycle – Deep Dive with Real Scenario
7. Android Project Structure & Manifest File
8. Emulators (AVD) & Debugging (DDMS / Logcat)
9. R.java & Resource Handling
10. Intents – Inter-Activity & Implicit Communication
11. Event Handlers & UI Interaction
12. Layouts, XML & the @ Sign
13. Adapters, Dynamic Lists & Holder Pattern (RecyclerView)
14. Menus and Dialogs
15. Services – Background Processing (with Real Example)
16. Data Storage – SharedPreferences, Files, SQLite
17. Web Service Integration – JSON, HTTP Clients (OkHttp)
18. Graphics, Widgets & Notifications
19. Multimedia & Telephony APIs
20. Android NDK (Native Development Kit)

**MOBILE APPLICATION DEVELOPMENT**

**A Comprehensive Textbook for BS IT 6th Semester (EI-333)**

**PART ONE: FOUNDATIONS**

# Chapter 1: Introduction to Mobile Computing

## 1.1 What is Mobile Computing?

Mobile computing refers to the ability to use computing devices (smartphones, tablets, wearables) while moving, without being tethered to a fixed network or power source. In simple words, it is computing on the go.

**Key characteristics that define mobile computing:**

- **Portability** – The devices are small, lightweight, and battery-operated. You can carry them in your pocket.
- **Wireless connectivity** – They use cellular networks (4G/5G), Wi-Fi, Bluetooth, or NFC to communicate.
- **Context awareness** – Built-in sensors (GPS, accelerometer, gyroscope, proximity sensor) let the device know where you are, how you are moving, and even what is around you.
- **Resource constraints** – Compared to desktop computers, mobile devices have limited battery life, less RAM, slower processors (to save power), and smaller storage.
- **Dynamic environment** – The network strength, available bandwidth, and even the device orientation can change at any moment.

**Real-life example:**
Imagine you are in a moving car, using Google Maps for navigation. The app uses GPS to track your location, downloads real-time traffic data over 4G, adjusts the screen orientation when you rotate your phone, and does all this while consuming as little battery as possible. If the network drops, it works offline. That is mobile computing at work.

## 1.2 The Mobile Computing Domain

The domain of mobile computing is vast. It includes:

- **Mobile application development** – Creating apps that run natively on a device (like Android apps written in Java/Kotlin).
- **Mobile web** – Websites that are responsive and work well on small screens.
- **Wearable computing** – Smartwatches, fitness trackers, smart glasses.

- **Internet of Things (IoT)** – Using a smartphone as a controller for smart lights, locks, and thermostats.
- **Mobile cloud computing** – Offloading heavy computation to cloud servers while using the phone as a thin client.

**Why is this important?**
Over 6 billion people use mobile phones worldwide. Almost every service – banking, education, healthcare, entertainment, shopping – has a mobile presence. Learning mobile development opens the door to a high-demand career.

## Chapter 2: Comparison of Mobile Platforms

### 2.1 The Major Players

When you build a mobile app, you have to choose a platform. The three dominant platforms today are:

1. **Android (Google)** – Open source, Linux-based, uses Java/Kotlin.
2. **iOS (Apple)** – Closed source, UNIX-based (Darwin), uses Swift/Objective-C.
3. **HarmonyOS (Huawei)** – Emerging, Linux + LiteOS, used mainly in China.

### 2.2 Detailed Comparison

| Feature | Android | iOS | HarmonyOS |
|---|---|---|---|
| **Kernel** | Linux | XNU (Darwin) | Linux + LiteOS |
| **Open source?** | Yes (AOSP) | No | Partial |
| **Official language** | Java, Kotlin | Swift, Objective-C | Java, JS, eTS |
| **IDE** | Android Studio | Xcode | DevEco Studio |
| **UI framework** | XML + Jetpack Compose | UIKit / SwiftUI | ArkUI |
| **App store** | Google Play + sideloading | App Store only | AppGallery |

| Feature | Android | iOS | HarmonyOS |
|---|---|---|---|
| **Customization** | High (launchers, ROMs) | Very low | Medium |
| **Global market share** | ~70% | ~28% | ~2% |
| **Developer account cost** | $25 one-time | $99/year | Free for now |
| **Hardware requirement** | Any PC/Mac | Mac only | Any PC |

### 2.3 Why Android for This Course?

- **You already know Java** – Android builds directly on Java (and now Kotlin). The prerequisite OOP course (CC-211) gives you the foundation.
- **It is free and open** – No need to buy a Mac. Android Studio runs on Windows, Linux, and Mac.
- **Large community** – Thousands of tutorials, stack overflow answers, and official documentation.
- **Job market** – Most mobile developer jobs (especially in Pakistan and India) are for Android.
- **Easy to test** – You can run apps on a real phone without paying any fee.

## Chapter 3: Revision of Java for Android

### 3.1 Why Java?

Android's original programming language is Java (though Kotlin is now preferred). The Android framework APIs are designed for Java. To be a good Android developer, you must understand the following Java concepts thoroughly.

### 3.2 Classes, Objects, and Inheritance

In Android, every screen is a class that extends Activity (or AppCompatActivity). For example:

java

```
public class MainActivity extends AppCompatActivity {
   // This class inherits all properties of AppCompatActivity
}
```

**What does inheritance give you?**
Your MainActivity automatically gets lifecycle methods (onCreate, onPause, etc.), the ability to start other activities, access to resources, and much more.

**Real example:** Just like a Car class extends Vehicle and gets startEngine() and stopEngine(), your ProfileActivity extends Activity and gets onCreate() and finish().

**3.3 Interfaces and Callbacks**

Interfaces are used heavily for event handling. The most common interface in Android is View.OnClickListener. When a button is tapped, the system calls the onClick method of the listener you provide.

java

```
button.setOnClickListener(new View.OnClickListener() {
   @Override
   public void onClick(View v) {
      // This is the callback – it runs when the button is tapped
   }
});
```

**Why interfaces?**
They allow the Android system to call YOUR code without knowing what you will do. This is called *inversion of control*.

**3.4 Collections Framework**

- **ArrayList** – Used to hold dynamic lists of objects, such as contacts, messages, or products.
- **HashMap** – Used for key-value pairs, e.g., storing user settings or JSON data.

**Example:**

java

```
ArrayList<String> contactNames = new ArrayList<>();
contactNames.add("Ali");
contactNames.add("Sara");
// Later, you give this list to an Adapter to display in a RecyclerView.
```

**3.5 Exception Handling**

Mobile apps run on unreliable networks and limited storage. You must handle exceptions to prevent crashes.

java

```
try {
   URL url = new URL("https://api.example.com/data");
   HttpURLConnection conn = (HttpURLConnection) url.openConnection();
} catch (IOException e) {
   // Show a friendly message to the user: "No internet connection"
   e.printStackTrace();
}
```

**Real example:** When your internet is off, a well-written app shows “Can’t reach server” instead of crashing.

**3.6 Multithreading**

The Android UI runs on a single thread (the “main thread”). If you do a long operation (network call, heavy database query) on the main thread, the app freezes and Android shows “Application Not Responding” (ANR). So you must move long tasks to background threads.

**Old way (deprecated but still in exams):** AsyncTask.
**Modern way:** Use Thread + Handler, or Kotlin coroutines, or WorkManager.

java

```
new Thread(() -> {
   // Do network call here
   runOnUiThread(() -> {
      // Update UI here
   });
}).start();
```

**3.7 Lambda Expressions (Java 8+)**

Lambda expressions make listener code shorter and cleaner. Instead of an anonymous inner class:

java

```
button.setOnClickListener(v -> {
   // This is a lambda. v is the view that was clicked.
   Toast.makeText(this, "Clicked!", Toast.LENGTH_SHORT).show();
});
```

Android supports many Java 8 features via a process called *desugaring*.

**PART TWO: ANDROID PLATFORM FUNDAMENTALS**

## Chapter 4: The Android Platform and Dalvik Virtual Machine

### 4.1 The Android Software Stack

Android is not just a single piece of software; it is a layered architecture. Each layer has a specific job.

**From bottom to top:**

1. **Linux Kernel**
    - Handles device drivers (display, camera, Wi-Fi).
    - Manages memory, processes, and power.
    - Provides security (user-based permission model).
2. **Hardware Abstraction Layer (HAL)**
    - Provides standard interfaces for hardware components.
    - For example, the camera HAL allows the Android framework to talk to any camera chip.
3. **Android Runtime (ART)**
    - Executes the app's bytecode (DEX format).
    - Replaced the older Dalvik VM.
    - Manages garbage collection and memory.
4. **Native C/C++ Libraries**
    - OpenGL ES (graphics), SQLite (database), WebKit (browser engine), etc.
    - Accessed via Java APIs through JNI (Java Native Interface).
5. **Java API Framework**
    - This is what you use as an Android developer.
    - Includes Activity Manager, Content Providers, Notification Manager, etc.
6. **System Apps**
    - Phone, Contacts, Browser, Settings, etc.
    - Your own apps sit at the same level as these.

### 4.2 Dalvik Virtual Machine (The Old Way)

Before Android 5.0 (Lollipop), apps ran on the Dalvik Virtual Machine. Key points:

- **Register-based** – Unlike the JVM which is stack-based, Dalvik used registers. This meant fewer instructions and better performance on mobile.
- **DEX bytecode** – Java .class files were converted to .dex (Dalvik Executable) format. A single .dex file could contain multiple classes, reducing overhead.
- **JIT (Just-In-Time) compilation** – The code was compiled to native machine code at runtime. Each time you opened an app, it would recompile parts of it.
- **Downside** – Slower first launch, more battery drain due to repeated compilation.

**4.3 ART (Android Runtime) – Current**

Starting with Android 5.0, ART (Android Runtime) replaced Dalvik. ART uses a hybrid approach:

- **AOT (Ahead-Of-Time) compilation** – During app installation, the entire app is compiled to native code. This makes the app launch faster and run smoother.
- **Downside** – Longer installation time and larger app size.
- **Android 7.0+ (Nougat) hybrid** – ART now uses a mix of AOT, JIT, and profile-guided compilation. It compiles only the frequently used parts, resulting in the best of both worlds.

**Real difference:**
On an old Dalvik device (Android 4.4), opening a heavy app like Facebook could take 3-4 seconds. On a modern ART device, it opens in under 1 second.

**4.4 Why Not Standard JVM?**

The standard Java Virtual Machine (JVM) was designed for desktops and servers. It assumes:

- Unlimited power supply.
- Large memory (hundreds of MB or GB).
- Stack-based execution (more instructions per operation).

Android needed something that:

- Runs on limited battery.
- Uses less memory.
- Starts quickly.

Dalvik/ART solved these by being register-based, using DEX format (smaller memory footprint), and optimizing for mobile processors.

## Chapter 5: Android Constructs – The Four Pillars

Every Android application is made up of one or more of these four fundamental components. They are declared in AndroidManifest.xml and are managed by the Android system.

**5.1 Activity – The UI Screen**

An Activity represents a single screen with a user interface. For example, in a WhatsApp-like app:

- LoginActivity – screen for entering phone number.
- ChatListActivity – screen showing all conversations.
- ChatDetailActivity – screen for a single conversation.

**Key points about Activity:**

- It has a lifecycle (created, started, resumed, paused, stopped, destroyed).
- It must be declared in the manifest with <activity> tag.
- One activity is marked as the launcher (main entry point).

**Real example:** When you tap on a contact in your phone's contacts app, a new Activity opens showing that contact's details. When you press the back button, that Activity is destroyed and you return to the list.

**5.2 Service – Background Processing Without UI**

A Service is a component that runs in the background to perform long-running operations or to perform work for remote processes. It has no user interface.

**Two types of services:**

- **Started service** – Runs until it completes its task or until you stop it explicitly. Example: downloading a large file.
- **Bound service** – Allows other components (like an Activity) to bind to it and call methods. Example: a music player service that the UI can control.

**Real example:** Spotify's music playback service continues playing even when you switch to another app or turn off the screen. The service shows a notification so you know it's still running.

**5.3 Content Provider – Sharing Data Between Apps**

A Content Provider manages a shared set of app data. It allows other apps (or the same app) to query, insert, update, or delete data using a standard interface (URIs).

**Built-in Content Providers:**

- ContactsContract – for the device's contacts.
- MediaStore – for images, videos, and audio files.
- CallLog – for call history.

**Real example:** Your WhatsApp app wants to show you a list of friends who already use WhatsApp. It reads your phone's contacts using the Contacts Content Provider. It does not need direct access to the contacts database; it uses the provider's query method.

### 5.4 Broadcast Receiver – Responding to System Events

A Broadcast Receiver is a component that listens for system-wide broadcast messages. Many broadcasts originate from the system (e.g., battery low, screen turned off, SMS received). Your app can also send custom broadcasts.

**Real example:** You install a battery saver app. It registers a Broadcast Receiver for ACTION_BATTERY_LOW. When the system sends that broadcast, the receiver wakes up and shows a notification suggesting you turn off Wi-Fi or reduce brightness.

**Another example:** An app that needs to start a service after the device finishes booting. It listens for BOOT_COMPLETED broadcast.

**Important:** Starting from Android 8.0 (Oreo), there are restrictions on implicit broadcasts. Many broadcasts no longer work for apps targeting newer APIs unless the app is running.

## Chapter 6: Activity Lifecycle in Depth

The Activity lifecycle is one of the most important concepts in Android development. If you do not understand it, your app will leak memory, crash, or lose user data.

### 6.1 The Seven Callback Methods

When an Activity transitions from one state to another, the system calls specific methods. You override these methods to perform appropriate actions.

| Method | When is it called? | What should you do here? |
|---|---|---|
| onCreate() | Activity is being created (first time) | Initialize UI, set content view, find views, restore saved state |
| onStart() | Activity becomes visible to the user | Prepare the UI for display (e.g., start animations) |
| onResume() | Activity starts interacting with the user | Start camera preview, resume video playback, register listeners |

| Method | When is it called? | What should you do here? |
|---|---|---|
| onPause() | Another activity gains focus (partial visible) | Pause animations, release camera, save unsaved data (like a draft email) |
| onStop() | Activity is no longer visible | Stop GPS updates, release heavy resources (bitmaps, database cursors) |
| onRestart() | Activity was stopped and is about to start again | Reinitialize anything that was released in onStop() |
| onDestroy() | Activity is finishing or system kills it | Unregister all listeners, close database connections, clean up |

**6.2 The Lifecycle in Practice – A Real Scenario**

Imagine you are using a note-taking app. You are typing a long note in the EditNoteActivity. Suddenly, your phone rings. The incoming call screen appears.

**What happens?**

1. The system calls onPause() on your activity. You save the partially typed note to SharedPreferences or a database. This is critical – if you don't save here and the system kills your app to free memory for the call, your user's note will be lost.
2. Then onStop() is called because your activity is no longer visible (the call screen covers the whole screen).
3. The user finishes the call and returns to your app.
4. The system calls onRestart() (since the activity was stopped, not destroyed), then onStart(), then onResume().
5. You reload the saved draft from SharedPreferences and show it in the text box. The user is happy because their note is still there.

**What if the user rotates the phone?**

- The current activity is destroyed and recreated because the screen configuration (orientation) changed.
- Sequence: onPause() → onStop() → onDestroy() → then a new instance goes through onCreate() → onStart() → onResume().
- To preserve data across rotation, you use onSaveInstanceState(Bundle).

### 6.3 Saving and Restoring Instance State

The Bundle object is like a suitcase that can hold primitive values and serializable objects.

**Saving:**

java

```
@Override
protected void onSaveInstanceState(Bundle outState) {
   super.onSaveInstanceState(outState);
   outState.putString("note_text", editText.getText().toString());
   outState.putInt("note_position", currentPosition);
}
```

**Restoring (in onCreate or onRestoreInstanceState):**

java

```
@Override
protected void onCreate(Bundle savedInstanceState) {
   super.onCreate(savedInstanceState);
   setContentView(R.layout.activity_edit_note);
   if (savedInstanceState != null) {
      String savedText = savedInstanceState.getString("note_text");
      editText.setText(savedText);
   }
}
```

**Why not just save everything in onPause?**
Because onSaveInstanceState is specifically for transient UI state (like the text in an EditText, scroll position). Persistent data (like the final note content) should go to a database in onPause() or onStop().

## Chapter 7: Android Project Structure and Manifest File

### 7.1 The Anatomy of an Android Project (in Android Studio)

When you create a new project, Android Studio generates a folder structure. Understanding each part is essential.

text

```
MyFirstApp/
├── app/
│   ├── src/
│   │   ├── main/
│   │   │   ├── java/                # Your Java/Kotlin source files
│   │   │   │   └── com/example/myfirstapp/
│   │   │   │       ├── MainActivity.java
│   │   │   │       └── ... other classes
│   │   │   ├── res/                 # Resources (layouts, images, strings)
│   │   │   │   ├── layout/
│   │   │   │   │   └── activity_main.xml
│   │   │   │   ├── drawable/        # Images (PNG, JPG, vector)
│   │   │   │   ├── mipmap/          # App icons (different densities)
│   │   │   │   ├── values/
│   │   │   │   │   ├── strings.xml
│   │   │   │   │   ├── colors.xml
│   │   │   │   │   └── dimens.xml
│   │   │   │   └── menu/            # XML files for option menus
│   │   │   ├── AndroidManifest.xml  # The app's blueprint
│   │   ├── test/                    # Unit tests (JUnit)
│   │   ├── androidTest/             # Instrumented tests
│   ├── build.gradle                 # Module-level build configuration
├── build.gradle                     # Project-level build configuration
└── gradle.properties                # Gradle settings
```

**7.2 The AndroidManifest.xml – The Blueprint of Your App**

Every Android app must have a file named AndroidManifest.xml in its root directory. The manifest tells the Android system about your app before it runs any code.

**What goes into the manifest?**

- **Package name** – The unique identifier for your app (e.g., com.example.myapp). Used by Google Play and the system.
- **Permissions** – What hardware or data your app needs. For example:

xml

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.CAMERA" />
```

- **Components** – All activities, services, broadcast receivers, and content providers must be declared.
- **Intent filters** – Tells the system what intents your component can respond to. For example, the MAIN action and LAUNCHER category make an activity the entry point.
- **Minimum and target SDK** – Which Android versions your app supports.

**A minimal manifest example:**

xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
  package="com.example.myfirstapp">

  <uses-permission android:name="android.permission.INTERNET" />

  <application
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/AppTheme">

    <activity android:name=".MainActivity">
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
      </intent-filter>
    </activity>
```

```
    <service android:name=".MyBackgroundService" />

    <receiver android:name=".MyBroadcastReceiver" />

  </application>
</manifest>
```

**Real consequence of forgetting the manifest:**
If you create a new Activity class but forget to add the <activity> tag in the manifest, your app will crash with a ActivityNotFoundException when you try to start it.

## Chapter 8: Emulators (AVD) and Debugging (DDMS)

### 8.1 Android Virtual Device (AVD) – The Emulator

An AVD is a virtual phone that runs on your computer. It allows you to test your app without a physical device.

**How to create an AVD (using AVD Manager in Android Studio):**

1. Choose a device definition (e.g., Pixel 6, 6.7" screen).
2. Select a system image (e.g., Android 14, API level 34).
3. Configure RAM, SD card size, and enable hardware acceleration (Intel HAXM or Windows Hypervisor) for speed.

**Types of emulators:**

- **Software emulation (slow)** – Simulates an ARM processor on your x86 PC. Very slow.
- **Hardware acceleration (fast)** – Uses your computer's CPU virtualization. Almost as fast as a real device.

**Real use:** Before publishing an app, developers test on multiple AVDs with different screen sizes and Android versions. For example, you can test your layout on a small phone (4.7") and on a large tablet (10").

### 8.2 DDMS (Dalvik Debug Monitor Server) – Legacy but Important

DDMS was a tool in Eclipse (old Android IDE) that provided debugging services. In Android Studio, its features are integrated into **Logcat**, **Profiler**, and **Device File Explorer**.

**What DDMS used to do (and how we do it now):**

- **View logs** → Use Logcat with filters.

- **Take screenshots** → Use Layout Inspector or the screenshot button in Device Explorer.
- **Simulate incoming calls/SMS** → Use the "Virtual Sensors" or "Phone" controls in the emulator.
- **Explore file system** (/data/data/package) → Use Device File Explorer (View → Tool Windows → Device File Explorer).
- **Monitor threads and heap** → Use Android Profiler (CPU, Memory, Network).

**8.3 Modern Debugging Tools in Android Studio**

- **Logcat** – Shows system messages and your custom logs. You add logs in your code using Log.d(), Log.e(), etc.

java

Log.d("MainActivity", "Button clicked. User is logged in: " + isLoggedIn);

Then in Logcat, you can filter by tag ("MainActivity"), log level (Debug, Error), or search text.

- **Breakpoints** – Click in the gutter next to a line number. When the app hits that line, execution pauses. You can inspect variable values, step through code, and even change values on the fly.
- **Layout Inspector** – See the complete view hierarchy of your running app. You can check margins, padding, and even see which views are overlapping.
- **Profiler** – Real-time graphs of CPU usage, memory allocation, network activity, and battery consumption. If your app feels slow, the profiler can show you the culprit.

**Real debugging story:**
You app crashes only when you rotate the screen. You set a breakpoint in onSaveInstanceState and another in onCreate. You notice that the savedInstanceState bundle is null in onCreate after rotation – you realize you forgot to call super.onSaveInstanceState(outState). That one line was the fix.

## Chapter 9: The R.java File and Resource Handling

**9.1 What is R.java?**

When you build your Android project, the aapt2 tool (Android Asset Packaging Tool) scans your res/ folder and generates a Java class named R (short for Resources). This class contains static integer constants for every resource you have.

**For example, if you have:**

- res/layout/activity_main.xml → R.layout.activity_main
- res/values/strings.xml with a string named app_name → R.string.app_name
- res/drawable/icon.png → R.drawable.icon

**You never modify R.java manually.** It is regenerated every time you change any resource. If you manually change it, your next build will overwrite your changes.

**9.2 Accessing Resources in Java Code**

java

```
// Set the layout
setContentView(R.layout.activity_main);


// Get a string resource
String hello = getString(R.string.hello_world);


// Get a drawable (image)
Drawable logo = getDrawable(R.drawable.logo);


// Get a color
int color = getColor(R.color.primary_blue);
```

**9.3 Accessing Resources in XML Layouts**

In your XML layout files, you use the @ symbol to reference resources.

xml

```
<TextView
   android:text="@string/welcome_message"
   android:textColor="@color/black"
   android:background="@drawable/rounded_button"
   android:id="@+id/myTextView" />
```

- @string/welcome_message – Look for the string with key welcome_message in strings.xml.
- @color/black – Built-in Android color.
- @drawable/rounded_button – A drawable resource (could be an XML shape or an image).
- @+id/myTextView – Create a new ID for this view. The + means “create if not exists”. Later in Java, you use R.id.myTextView.

**9.4 Resource Types and Their Folders**

| Folder | What goes inside |
|---|---|
| res/layout/ | XML files that define UI layouts (e.g., activity_main.xml) |
| res/drawable/ | Images (PNG, JPG, GIF, Vector drawables), and XML drawables (shapes, selectors) |
| res/mipmap/ | App icons at different densities (hdpi, xhdpi, xxhdpi, etc.) |
| res/values/ | XML files with simple values: strings.xml, colors.xml, dimens.xml, styles.xml |
| res/menu/ | XML files for options menus, context menus |
| res/raw/ | Arbitrary files (e.g., audio files, video files) that are kept as-is |
| res/anim/ | XML animations (translate, rotate, scale) |

**Why separate resources?**
It allows Android to pick the best resource for the current device configuration. For example, you can have drawable-hdpi/icon.png for high-density screens and drawable-ldpi/icon.png for low-density screens. At runtime, Android automatically loads the correct one.

## Chapter 10: Inter-Activity Communication with Intents

### 10.1 What is an Intent?

An Intent is a messaging object that you can use to request an action from another app component. Think of it as a **postcard** that says "I want to do this action" or "I want to open that component".

**Intents are used for three main purposes:**

1. Starting an Activity.
2. Starting a Service.
3. Delivering a broadcast (to a BroadcastReceiver).

### 10.2 Explicit Intent – Directly Naming the Target

An explicit intent specifies exactly which component (by class name) should handle it. You use it to start an Activity within your own app.

**Example – moving from MainActivity to SecondActivity:**

java

```
Intent intent = new Intent(MainActivity.this, SecondActivity.class);

startActivity(intent);
```

**Passing data with explicit intent:**

java

```
Intent intent = new Intent(MainActivity.this, ProfileActivity.class);

intent.putExtra("user_name", "Ali Raza");

intent.putExtra("user_age", 22);

intent.putExtra("is_premium", true);

startActivity(intent);
```

**Receiving the data in ProfileActivity:**

java

```
String name = getIntent().getStringExtra("user_name");

int age = getIntent().getIntExtra("user_age", 0); // default 0 if not found

boolean isPremium = getIntent().getBooleanExtra("is_premium", false);
```

**10.3 Implicit Intent – Letting the System Find a Component**

An implicit intent does not name a specific component. Instead, it declares an action and optionally data. The system then finds the best app that can handle that action.

**Example – open a web page:**

java

```
Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse("https://www.google.com"));

startActivity(intent);
```

When this runs, Android shows a chooser dialog with all browsers installed (Chrome, Firefox, Edge, etc.). The user picks one.

**Example – pick a contact from the address book:**

java

```
Intent intent = new Intent(Intent.ACTION_PICK, ContactsContract.Contacts.CONTENT_URI);

startActivityForResult(intent, PICK_CONTACT_REQUEST);
```

**Example – share text (like “Share” button):**

java

```
Intent intent = new Intent(Intent.ACTION_SEND);
intent.setType("text/plain");
intent.putExtra(Intent.EXTRA_TEXT, "Check out this amazing app!");
startActivity(Intent.createChooser(intent, "Share via"));
```

**10.4 Starting an Activity for a Result (with data returned)**

Sometimes you want to start a second activity and get a result back (e.g., pick a contact, capture a photo, ask for a text input). The old method (startActivityForResult) is deprecated, but you still see it in legacy code. The modern way uses ActivityResultLauncher.

**Modern example (API 30+):**

java

```
// Declare a launcher
ActivityResultLauncher<Intent> someActivityResultLauncher = registerForActivityResult(
    new ActivityResultContracts.StartActivityForResult(),
    result -> {
        if (result.getResultCode() == RESULT_OK) {
            Intent data = result.getData();
            String returnedText = data.getStringExtra("result");
            // Do something with the result
        }
    });

// Use it to start an activity
Intent intent = new Intent(this, InputActivity.class);
someActivityResultLauncher.launch(intent);
```

In the second activity, you call setResult() before finishing:

java

```
Intent resultIntent = new Intent();
```

```
resultIntent.putExtra("result", userInput);

setResult(RESULT_OK, resultIntent);

finish();
```

## Chapter 11: Event Handlers and UI Interaction

### 11.1 The Event Listener Model

In Android, when a user interacts with a view (tapping, long pressing, touching, typing), the system generates an event. You can respond to that event by registering a **listener**.

The most common pattern is:

1. Find the view (e.g., a Button) using findViewById().
2. Call the view's setOn...Listener() method, passing an object that implements the corresponding listener interface.
3. Implement the callback method (e.g., onClick()).

### 11.2 Setting an OnClickListener – Three Ways

**Way 1: Anonymous inner class (most common in learning)**

java

```
Button btn = findViewById(R.id.submitButton);

btn.setOnClickListener(new View.OnClickListener() {

   @Override

   public void onClick(View v) {

      // This code runs when the button is tapped

      Toast.makeText(MainActivity.this, "Submitted!", Toast.LENGTH_SHORT).show();

   }

});
```

**Way 2: Lambda expression (requires Java 8)**

java

```
btn.setOnClickListener(v -> {

   Toast.makeText(this, "Submitted!", Toast.LENGTH_SHORT).show();
```

```
});
```

**Way 3: Activity implements the listener interface**

java

```
public class MainActivity extends AppCompatActivity implements View.OnClickListener {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Button btn = findViewById(R.id.submitButton);
        btn.setOnClickListener(this);  // "this" refers to the activity
    }

    @Override
    public void onClick(View v) {
        // Handle click based on view's ID
        if (v.getId() == R.id.submitButton) {
            // Do something
        }
    }
}
```

**11.3 Other Important Event Listeners**

- **OnLongClickListener** – Triggers when the user presses and holds a view. Used for showing context menus or deleting items.

java

```
view.setOnLongClickListener(v -> {
    // Show delete dialog
    return true; // return true to consume the event
});
```

- **OnTouchListener** – Gives raw touch coordinates (down, move, up). Used for drawing apps, custom gestures, or drag-and-drop.

```
java

view.setOnTouchListener((v, event) -> {
   float x = event.getX();
   float y = event.getY();
   int action = event.getAction();
   // ACTION_DOWN, ACTION_MOVE, ACTION_UP
   return true;
});
```

- **OnKeyListener** – For hardware keys (volume, power, back). Rarely used because most devices have software keys.

**Real example:** In a drawing app, you use OnTouchListener to capture finger movements and draw lines on a custom canvas.

## Chapter 12: Layouts, XML, and the @ Sign

### 12.1 What is a Layout?

A layout is a ViewGroup (a container that holds other views) that defines the visual structure of the user interface. Layouts are typically defined in XML files inside res/layout/. Each XML file corresponds to one screen (Activity) or a reusable portion of a screen (Fragment, custom view).

### 12.2 The Five Most Common Layouts

| Layout | Behaviour | When to use |
|---|---|---|
| **LinearLayout** | Arranges children in a single row (horizontal) or single column (vertical). | Simple forms, sequential items. |
| **RelativeLayout** | Positions children relative to parent or to each other (e.g., layout_below, layout_toRightOf). | Complex overlapping, but newer ConstraintLayout is better. |

| Layout | Behaviour | When to use |
|---|---|---|
| **ConstraintLayout** | Uses constraints (like springs) to position children. Flat hierarchy, very flexible. | Almost all modern apps. Recommended for complex designs. |
| **FrameLayout** | Only one child at a time. | Fragments, loading overlays, simple placeholders. |
| **GridLayout** | Places children in a grid (rows and columns). | Calculator buttons, photo gallery. |

**12.3 XML Layout Example (ConstraintLayout)**

xml

```
<androidx.constraintlayout.widget.ConstraintLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:app="http://schemas.android.com/apk/res-auto"
   android:layout_width="match_parent"
   android:layout_height="match_parent">

   <Button
      android:id="@+id/loginButton"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Login"
      app:layout_constraintTop_toTopOf="parent"
      app:layout_constraintLeft_toLeftOf="parent"
      app:layout_constraintRight_toRightOf="parent"
      app:layout_constraintBottom_toBottomOf="parent" />

</...>
```

**Explanation:**

- match_parent – The view takes all available space from its parent.
- wrap_content – The view sizes itself to fit its content (e.g., the text inside the button).
- Constraints (e.g., layout_constraintTop_toTopOf="parent") – The button's top edge is aligned with the parent's top edge. The same for left, right, bottom – this centers the button both horizontally and vertically.

### 12.4 Understanding the @ Sign in XML

The @ symbol tells the Android resource system to look up a resource.

| Syntax | Meaning | Example |
|---|---|---|
| @string/name | Reference a string from strings.xml | android:text="@string/welcome" |
| @color/name | Reference a color from colors.xml | android:textColor="@color/primary" |
| @drawable/name | Reference an image or drawable XML | android:src="@drawable/logo" |
| @+id/name | Create a new ID for a view | android:id="@+id/myButton" |
| @android:color/black | Use Android's built-in resources | android:textColor="@android:color/black" |
| @style/StyleName | Reference a style from styles.xml | style="@style/TitleText" |

**Real example of** @+id**:**
When you write android:id="@+id/emailInput", you are telling Android: "Create a new integer constant called emailInput in the R.id class." Later in Java, you can write findViewById(R.id.emailInput).

## Chapter 13: Adapters, Dynamic Lists, and the Holder Pattern

### 13.1 What is an Adapter?

An **Adapter** is an object that acts as a bridge between a data source (e.g., an array or a database cursor) and a view that displays a list of items (like ListView or RecyclerView). The adapter creates the child views for each item and fills them with data.

**Think of it as:**

- You have a list of 100 names (data).
- You have a blank template (the item layout).
- The adapter takes each name, puts it into a copy of the template, and hands that view to the list.

**13.2 RecyclerView – The Modern Standard**

RecyclerView is the successor to ListView. It is more flexible, more performant, and encourages the **ViewHolder pattern**.

**Steps to implement a RecyclerView:**

1. **Add the dependency** in build.gradle (though it's usually included by default in new projects).
2. **Add** RecyclerView **widget** to your activity's XML layout.
3. **Create an item layout** (e.g., item_contact.xml) – this defines how each row looks.
4. **Create a ViewHolder class** – a small class that holds references to the views inside one item layout.
5. **Create an Adapter class** that extends RecyclerView.Adapter<YourViewHolder>.
6. **Implement three methods**:
    - onCreateViewHolder() – inflates the item layout and creates a ViewHolder.
    - onBindViewHolder() – populates the ViewHolder with data at a given position.
    - getItemCount() – returns the total number of items.
7. **Set the adapter** on the RecyclerView.

**13.3 The Holder Pattern Explained**

The **ViewHolder pattern** is an optimization. Without it, findViewById() would be called every time a row scrolls into view, causing lag. With the pattern, you call findViewById() once when the row is first created, store the references in a ViewHolder object, and reuse that object for all future bindings.

**Example of a ViewHolder:**

java

```
public static class ContactViewHolder extends RecyclerView.ViewHolder {
    TextView nameTextView;
    TextView phoneTextView;
    ImageView profileImageView;
```

```
    public ContactViewHolder(@NonNull View itemView) {
        super(itemView);
        nameTextView = itemView.findViewById(R.id.contact_name);
        phoneTextView = itemView.findViewById(R.id.contact_phone);
        profileImageView = itemView.findViewById(R.id.contact_image);
    }
}
```

**Adapter implementation skeleton:**

java

```
public class ContactAdapter extends RecyclerView.Adapter<ContactAdapter.ContactViewHolder> {

    private List<Contact> contactList;

    public ContactAdapter(List<Contact> contacts) {
        this.contactList = contacts;
    }

    @NonNull
    @Override
    public ContactViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(parent.getContext())
                .inflate(R.layout.item_contact, parent, false);
        return new ContactViewHolder(view);
    }

    @Override
    public void onBindViewHolder(@NonNull ContactViewHolder holder, int position) {
        Contact contact = contactList.get(position);
        holder.nameTextView.setText(contact.getName());
```

```
        holder.phoneTextView.setText(contact.getPhone());
        // Set image, etc.
    }

    @Override
    public int getItemCount() {
        return contactList.size();
    }

    // Inner ViewHolder class
    static class ContactViewHolder extends RecyclerView.ViewHolder { ... }
}
```

**13.4 Dynamic Lists (Adding, Removing, Updating)**

When your data changes (e.g., user adds a new contact), you must notify the adapter. The adapter will then update only the affected rows.

```
java
// Add a new contact at the end
contactList.add(newContact);
adapter.notifyItemInserted(contactList.size() - 1);

// Remove an item
int position = // ... get position
contactList.remove(position);
adapter.notifyItemRemoved(position);

// Update entire list (less efficient)
contactList.clear();
contactList.addAll(newList);
adapter.notifyDataSetChanged();
```

**Real example:** WhatsApp's chat list. When a new message arrives, only that chat row is updated (the notifyItemChanged(position) method is called). That's why the list scrolls smoothly even with thousands of chats.

## Chapter 14: Menus and Dialogs

### 14.1 Menus in Android

Android supports three types of menus:

| Menu Type | How it appears | Typical use |
|---|---|---|
| **Options Menu** | In the app bar (toolbar) – the three dots | Settings, search, refresh, etc. |
| **Context Menu** | Appears on long-press of a view | Edit, delete, copy, paste |
| **Popup Menu** | Anchored to a specific view (short tap) | Additional actions attached to a button |

### 14.2 Creating an Options Menu via XML

First, create a menu XML file in res/menu/, e.g., main_menu.xml:

xml

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">
   <item
      android:id="@+id/action_settings"
      android:title="Settings"
      android:icon="@drawable/ic_settings"
      android:showAsAction="ifRoom" />
   <item
      android:id="@+id/action_share"
      android:title="Share"
      android:icon="@drawable/ic_share"
      android:showAsAction="ifRoom" />
   <item
```

```
        android:id="@+id/action_about"
        android:title="About" />
</menu>
```

Then, in your Activity, override:

java

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
   getMenuInflater().inflate(R.menu.main_menu, menu);
   return true;
}

@Override
public boolean onOptionsItemSelected(MenuItem item) {
   int id = item.getItemId();
   if (id == R.id.action_settings) {
      openSettings();
      return true;
   } else if (id == R.id.action_share) {
      shareApp();
      return true;
   } else if (id == R.id.action_about) {
      showAboutDialog();
      return true;
   }
   return super.onOptionsItemSelected(item);
}
```

**14.3 Context Menu (Long Press Menu)**

Register a view for context menu:

java

registerForContextMenu(myListView);

Then override:

java

```
@Override
public void onCreateContextMenu(ContextMenu menu, View v, ContextMenu.ContextMenuInfo menuInfo) {
   super.onCreateContextMenu(menu, v, menuInfo);
   getMenuInflater().inflate(R.menu.context_menu, menu);
   menu.setHeaderTitle("Choose action");
}


@Override
public boolean onContextItemSelected(MenuItem item) {
   // Get info about which item was long-pressed
   AdapterView.AdapterContextMenuInfo info = (AdapterView.AdapterContextMenuInfo) item.getMenuInfo();
   int position = info.position;
   // Handle the action based on item ID
   return super.onContextItemSelected(item);
}
```

**14.4 Dialogs**

A dialog is a small window that appears in front of the current activity, asking the user to make a decision or enter information.

**AlertDialog – the most common type:**

java

```
new AlertDialog.Builder(this)
   .setTitle("Confirm Delete")
   .setMessage("Are you sure you want to delete this message?")
   .setPositiveButton("Delete", (dialog, which) -> {
      // Delete the message
```

```
    })
    .setNegativeButton("Cancel", null)
    .setNeutralButton("Remind later", (dialog, which) -> {
        // Postpone
    })
    .show();
```

**DatePickerDialog – for picking a date:**

```
java
Calendar calendar = Calendar.getInstance();
int year = calendar.get(Calendar.YEAR);
int month = calendar.get(Calendar.MONTH);
int day = calendar.get(Calendar.DAY_OF_MONTH);

DatePickerDialog datePicker = new DatePickerDialog(this,
    (view, selectedYear, selectedMonth, selectedDay) -> {
        // Use the selected date
    }, year, month, day);
datePicker.show();
```

**ProgressDialog is deprecated.** Instead, use a ProgressBar inside a dialog or use AlertDialog with a custom view.

**Real example:** When you try to close a document without saving, a dialog appears: “Save changes?” with buttons “Save”, “Discard”, and “Cancel”. That is an AlertDialog.

## Chapter 15: Services – Background Processing

### 15.1 What is a Service?

A **Service** is an application component that can perform long-running operations in the background without providing a user interface. For example, a music player app might use a service to keep playing music even after you leave the app.

**Important:** By default, a service runs on the **main thread**. If you do heavy work inside a service without creating a background thread, you will get ANR (Application Not Responding).

**15.2 Types of Services (Android 8+)**

| Type | How it runs | Required notification? | Use case |
|---|---|---|---|
| **Foreground Service** | User is aware (notification visible) | Yes | Music playback, GPS tracking, file download |
| **Background Service** | Runs without notification, but very limited time | No (but restricted) | Short tasks (few seconds) – for anything longer, use WorkManager |
| **Bound Service** | Allows components to bind and call methods | No | Client-server scenario (e.g., a service that provides an API to the UI) |

**15.3 Creating a Simple Started Service**

java

```
public class MyDownloadService extends Service {

  @Override
  public void onCreate() {
    super.onCreate();
    // Called once when service is first created
  }

  @Override
  public int onStartCommand(Intent intent, int flags, int startId) {
    // This runs every time startService() is called
    // Do your background work here (but on a separate thread!)

    // If you want the service to restart if killed by system, return START_STICKY
```

```
        return START_STICKY;
    }

    @Override
    public void onDestroy() {
        super.onDestroy();
        // Clean up
    }

    @Nullable
    @Override
    public IBinder onBind(Intent intent) {
        return null; // Not a bound service
    }
}
```

**Starting the service from an Activity:**

java

```
Intent serviceIntent = new Intent(this, MyDownloadService.class);
startService(serviceIntent);
```

**Stopping the service:**

java

```
stopService(serviceIntent); // from outside
// or inside the service call stopSelf()
```

**15.4 Foreground Service Example (with Notification)**

For Android 8+, any service that runs for more than a few seconds must be a foreground service (or use WorkManager). Foreground service must show a notification.

java

```
@Override
public int onStartCommand(Intent intent, int flags, int startId) {
```

```
    // Create a notification
    Notification notification = new NotificationCompat.Builder(this, "CHANNEL_ID")
        .setContentTitle("Downloading")
        .setContentText("Downloading file...")
        .setSmallIcon(R.drawable.ic_download)
        .build();

    // Start foreground mode
    startForeground(1, notification);

    // Do the download (on background thread)
    return START_STICKY;
}
```

**15.5 The Modern Alternative: WorkManager**

For deferrable, guaranteed background tasks (like syncing data every 24 hours, uploading logs), Google recommends using **WorkManager** instead of services. WorkManager automatically chooses the best way to run the task (JobScheduler, AlarmManager, or a foreground service) depending on the Android version.

**Why not just use Service always?**
Because Android kills background services aggressively to save battery. WorkManager is designed to handle these restrictions.

## Chapter 16: Data Storage – SharedPreferences, Files, SQLite

**16.1 SharedPreferences – Simple Key-Value Storage**

SharedPreferences is the simplest way to store small amounts of primitive data (boolean, int, float, long, String, Set<String>). It is stored as an XML file in the app's private directory.

**When to use?**
User settings, game high scores, last login time, user's preference (dark mode on/off).

**Writing to SharedPreferences:**

java

```
// Get preferences file (creates if not exists)
SharedPreferences prefs = getSharedPreferences("MyAppPrefs", MODE_PRIVATE);
SharedPreferences.Editor editor = prefs.edit();
editor.putString("username", "ali_raza");
editor.putBoolean("dark_mode", true);
editor.putInt("high_score", 2500);
editor.apply(); // asynchronous commit (preferred)
// or editor.commit(); // synchronous (blocks UI thread)
```

**Reading from SharedPreferences:**

java

```
SharedPreferences prefs = getSharedPreferences("MyAppPrefs", MODE_PRIVATE);
String username = prefs.getString("username", "default_user");
boolean darkMode = prefs.getBoolean("dark_mode", false);
int highScore = prefs.getInt("high_score", 0);
```

**16.2 Internal File Storage**

You can store arbitrary files (text, images, binary) in the app's internal storage directory. These files are private to your app and are deleted when the app is uninstalled.

**Writing a file:**

java

```
String fileName = "data.txt";
String content = "Hello, this is my secret data.";
try (FileOutputStream fos = openFileOutput(fileName, MODE_PRIVATE)) {
    fos.write(content.getBytes());
} catch (IOException e) {
    e.printStackTrace();
}
```

**Reading a file:**

java

```
try (FileInputStream fis = openFileInput("data.txt")) {
```

```
    InputStreamReader isr = new InputStreamReader(fis);
    BufferedReader br = new BufferedReader(isr);
    String text = br.readLine();
    // use the text
} catch (IOException e) {
    e.printStackTrace();
}
```

**16.3 External Storage (SD Card, Shared Storage)**

External storage (like the device's "Downloads" folder or a real SD card) can be accessed by other apps and the user. You need permissions:

- READ_EXTERNAL_STORAGE
- WRITE_EXTERNAL_STORAGE (for Android 10 and below; Android 10+ has scoped storage)

**Getting external storage directory:**

java

```
File externalDir = getExternalFilesDir(null); // app-specific external directory
// This directory is removed when the app is uninstalled (good for temporary files)
```

**16.4 SQLite Database – Full Relational Database**

For structured data (like a table of contacts, messages, or products), use SQLite. Android includes a built-in SQLite database.

**Key classes:**

- SQLiteOpenHelper – manages database creation and version upgrades.
- SQLiteDatabase – provides methods for CRUD operations.
- Cursor – holds the result of a query (like a pointer to a row set).

**Creating a Database Helper:**

java

```
public class MyDatabaseHelper extends SQLiteOpenHelper {

    private static final String DATABASE_NAME = "myapp.db";
    private static final int DATABASE_VERSION = 1;
```

```
    public MyDatabaseHelper(Context context) {
        super(context, DATABASE_NAME, null, DATABASE_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        String createTable = "CREATE TABLE users (" +
                "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
                "name TEXT, " +
                "email TEXT UNIQUE, " +
                "age INTEGER)";
        db.execSQL(createTable);
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS users");
        onCreate(db);
    }
}
```

**Inserting a record:**

```
java
MyDatabaseHelper helper = new MyDatabaseHelper(this);
SQLiteDatabase db = helper.getWritableDatabase();
ContentValues values = new ContentValues();
values.put("name", "Ali Raza");
values.put("email", "ali@example.com");
values.put("age", 25);
```

```
long newRowId = db.insert("users", null, values);
```

**Querying records:**

```
java

SQLiteDatabase db = helper.getReadableDatabase();

Cursor cursor = db.query("users",

   new String[]{"id", "name", "email", "age"},

   "age > ?", new String[]{"18"}, // selection, selectionArgs

   null, null, "name ASC");


while (cursor.moveToNext()) {

   int id = cursor.getInt(0);

   String name = cursor.getString(1);

   String email = cursor.getString(2);

   int age = cursor.getInt(3);

}

cursor.close();

db.close();
```

**Real example:** WhatsApp stores all your messages in an SQLite database (encrypted). When you open a chat, the app queries the database for messages from that contact and displays them in the RecyclerView.

## Chapter 17: Web Service Integration – JSON and HTTP Clients

### 17.1 The Need for Web Services

Modern mobile apps rarely work alone. They talk to servers on the internet to fetch data (news, weather, posts, products) or send data (user registration, orders, comments). The most common data format for these APIs is **JSON** (JavaScript Object Notation).

**Example JSON response from a weather API:**

```
json

{
```

```
  "city": "Lahore",
  "temperature": 32,
  "unit": "Celsius",
  "forecast": ["Sunny", "Humid", "Partly Cloudy"]
}
```

**17.2 Parsing JSON in Android**

Android has built-in classes: JSONObject and JSONArray.

**Parsing the above JSON:**

```
java
String jsonString = "{\"city\":\"Lahore\",\"temperature\":32}";
try {
    JSONObject obj = new JSONObject(jsonString);
    String city = obj.getString("city");         // "Lahore"
    int temp = obj.getInt("temperature");        // 32
} catch (JSONException e) {
    e.printStackTrace();
}
```

**Parsing an array:**

```
java
String jsonArrayString = "[{\"name\":\"Ali\"},{\"name\":\"Sara\"}]";
JSONArray array = new JSONArray(jsonArrayString);
for (int i = 0; i < array.length(); i++) {
    JSONObject obj = array.getJSONObject(i);
    String name = obj.getString("name");
}
```

**17.3 Making HTTP Requests – The Old Way (HttpURLConnection)**

HttpURLConnection is the built-in Java class for HTTP. It works but is verbose.

```
java
URL url = new URL("https://api.weatherapi.com/v1/current.json?key=YOUR_KEY&q=Lahore");
```

```
HttpURLConnection conn = (HttpURLConnection) url.openConnection();
conn.setRequestMethod("GET");
conn.connect();

int responseCode = conn.getResponseCode();
if (responseCode == HttpURLConnection.HTTP_OK) {
   InputStream in = conn.getInputStream();
   BufferedReader reader = new BufferedReader(new InputStreamReader(in));
   StringBuilder response = new StringBuilder();
   String line;
   while ((line = reader.readLine()) != null) {
      response.append(line);
   }
   // response.toString() contains the JSON
}
conn.disconnect();
```

**Important:** This must run on a background thread (not the UI thread). Otherwise, the app will freeze and crash.

**17.4 Modern Approach: OkHttp and Retrofit**

Most professional Android apps use **OkHttp** (by Square) and **Retrofit** for networking. They are not built-in, but you can add them as dependencies in build.gradle.

**OkHttp example (simpler than HttpURLConnection):**

java

```
OkHttpClient client = new OkHttpClient();
Request request = new Request.Builder()
   .url("https://api.weatherapi.com/v1/current.json?key=KEY&q=Lahore")
   .build();

client.newCall(request).enqueue(new Callback() {
```

```
    @Override
    public void onFailure(Call call, IOException e) {
        // Network error
    }

    @Override
    public void onResponse(Call call, Response response) throws IOException {
        String json = response.body().string();
        // Parse JSON on a background thread, then update UI via runOnUiThread
    }
});
```

**Real example:** When you open the Daraz app, it makes an HTTP request to its server to fetch product listings, parses the JSON response, and displays the list.

## Chapter 18: Graphics, Widgets, and Notifications

### 18.1 Graphics in Android

Android supports three types of graphics:

1. **Drawables** – Images (PNG, JPG, SVG vector) or XML-defined shapes.
2. **Canvas & Paint** – For custom drawing (draw lines, circles, text) inside a View's onDraw() method.
3. **Animations** – Move, rotate, scale, fade views over time.

**Custom drawing example (a simple drawing view):**

java

```
public class MyCanvasView extends View {
    private Paint paint;
    private Path path;

    public MyCanvasView(Context context) {
        super(context);
```

```
        paint = new Paint();
        paint.setColor(Color.RED);
        paint.setStrokeWidth(10);
        paint.setStyle(Paint.Style.STROKE);
        path = new Path();
    }

    @Override
    protected void onDraw(Canvas canvas) {
        super.onDraw(canvas);
        canvas.drawPath(path, paint);
    }

    @Override
    public boolean onTouchEvent(MotionEvent event) {
        float x = event.getX();
        float y = event.getY();
        if (event.getAction() == MotionEvent.ACTION_DOWN) {
            path.moveTo(x, y);
        } else if (event.getAction() == MotionEvent.ACTION_MOVE) {
            path.lineTo(x, y);
        }
        invalidate(); // redraw
        return true;
    }
}
```

**18.2 App Widgets (Home Screen Widgets)**

An **App Widget** is a mini application that can be placed on the home screen (like a clock, weather widget, music controller).

**To create a widget, you need:**

- An AppWidgetProvider class (extends BroadcastReceiver).
- An XML layout for the widget.
- An XML definition file (widget_info.xml) with update frequency, size, etc.
- Declaration in the manifest.

Because this is an advanced topic, the course introduces it briefly.

**18.3 Notifications**

A notification is a message that appears outside your app's UI (in the status bar, or as a heads-up popup). It's how apps get the user's attention when they are not actively using the app.

**Creating a notification (Android 8+ requires a channel):**

java

```
// 1. Create a notification channel (only once, e.g., in onCreate of your app)
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
   NotificationChannel channel = new NotificationChannel(
      "CHANNEL_ID",
      "My Channel Name",
      NotificationManager.IMPORTANCE_DEFAULT);
   NotificationManager manager = getSystemService(NotificationManager.class);
   manager.createNotificationChannel(channel);
}


// 2. Build the notification
NotificationCompat.Builder builder = new NotificationCompat.Builder(this, "CHANNEL_ID")
   .setSmallIcon(R.drawable.ic_notification)
   .setContentTitle("New Message")
   .setContentText("You have a new message from Ali")
   .setPriority(NotificationCompat.PRIORITY_DEFAULT)
   .setAutoCancel(true); // remove when tapped
```

// 3. Show it

NotificationManagerCompat manager = NotificationManagerCompat.from(this);

manager.notify(1, builder.build());

**Tapping the notification can open an Activity:**

java

Intent intent = new Intent(this, MainActivity.class);

PendingIntent pendingIntent = PendingIntent.getActivity(this, 0, intent,
PendingIntent.FLAG_UPDATE_CURRENT);

builder.setContentIntent(pendingIntent);

**Real example:** When you receive a WhatsApp message while your phone is locked, a notification appears on the lock screen. Tapping it opens WhatsApp directly to that chat.

## Chapter 19: Multimedia and Telephony APIs

### 19.1 Playing Audio and Video

**MediaPlayer** is the primary class for audio/video playback.

**Playing a local sound file:**

java

MediaPlayer mediaPlayer = MediaPlayer.create(this, R.raw.notification_sound);

mediaPlayer.start();

**Playing a remote audio stream (e.g., internet radio):**

java

MediaPlayer mediaPlayer = new MediaPlayer();

mediaPlayer.setAudioStreamType(AudioManager.STREAM_MUSIC);

mediaPlayer.setDataSource("https://example.com/song.mp3");

mediaPlayer.prepareAsync(); // don't block UI thread

mediaPlayer.setOnPreparedListener(mp -> mp.start());

**Video playback using VideoView:**

xml

```
<VideoView
   android:id="@+id/videoView"
   android:layout_width="match_parent"
   android:layout_height="wrap_content" />
```

```
java
VideoView videoView = findViewById(R.id.videoView);
videoView.setVideoPath("android.resource://" + getPackageName() + "/" + R.raw.my_video);
videoView.start();
```

**19.2 Camera API**

The modern way to use camera is **CameraX** (Jetpack library). It simplifies capturing photos and videos.

**Basic photo capture with CameraX:**

```
java
// In your Activity/Fragment
ImageCapture imageCapture = new ImageCapture.Builder().build();

// Bind camera lifecycle to the lifecycle owner
Camera camera = cameraProvider.bindToLifecycle(lifecycleOwner, cameraSelector, preview,
imageCapture);

// When user taps capture button
imageCapture.takePicture(ContextCompat.getMainExecutor(this), new
ImageCapture.OnImageCapturedCallback() {
   @Override
   public void onCaptureSuccess(ImageProxy image) {
      // Save the image
   }
});
```

**Note:** Camera requires CAMERA permission and runtime permission request.

**19.3 Telephony API**

The telephony package allows you to access phone hardware information, make calls, send SMS, and listen to phone state.

**Make a phone call (direct, with permission):**

java

Intent callIntent = new Intent(Intent.ACTION_CALL, Uri.parse("tel:03001234567"));

startActivity(callIntent);

Requires android.permission.CALL_PHONE.

**Get device information (TelephonyManager):**

java

TelephonyManager tm = (TelephonyManager) getSystemService(TELEPHONY_SERVICE);

String networkOperator = tm.getNetworkOperatorName(); // "Mobilink", "Telenor", etc.

String deviceId = tm.getDeviceId(); // IMEI (requires permission)

**Send an SMS:**

java

SmsManager smsManager = SmsManager.getDefault();

smsManager.sendTextMessage("03001234567", null, "Hello from my app!", null, null);

Requires android.permission.SEND_SMS.

**Real example:** A “Find My Phone” app might use telephony API to detect when a different SIM card is inserted.

## Chapter 20: Android Native Development Kit (NDK)

### 20.1 What is the NDK?

The **Native Development Kit (NDK)** is a toolset that allows you to write parts of your Android app in native languages like C and C++. Normally, Android apps run in the ART virtual machine (Java/Kotlin). With NDK, you can compile C/C++ code into native machine code libraries (.so files) and call them from your Java code using **JNI (Java Native Interface)**.

### 20.2 When to Use the NDK?

- **Performance-critical code** – Games, signal processing, complex physics engines.
- **Reusing existing C/C++ libraries** – e.g., OpenCV (computer vision), FFmpeg (video processing), cryptography libraries.

- **Low-level hardware access** – Sometimes needed for custom drivers or embedded systems.

**Do not use NDK for typical app code.** It adds complexity, increases app size, and makes debugging harder. Google recommends using Java/Kotlin unless you have a strong reason.

**20.3 A Simple NDK Example**

**Step 1:** Declare a native method in Java.

java

```
public class NativeLib {
   static {
      System.loadLibrary("native-lib"); // loads libnative-lib.so
   }
   public native String stringFromJNI();
}
```

**Step 2:** Generate C/C++ header or write the implementation directly.

Create a file native-lib.cpp:

cpp

```
#include <jni.h>
#include <string>

extern "C" JNIEXPORT jstring JNICALL
Java_com_example_myapp_NativeLib_stringFromJNI(JNIEnv* env, jobject /* this */) {
   std::string hello = "Hello from C++";
   return env->NewStringUTF(hello.c_str());
}
```

**Step 3:** Call it from Java.

java

```
NativeLib lib = new NativeLib();
String result = lib.stringFromJNI();
textView.setText(result); // displays "Hello from C++"
```

**20.4 Setting Up NDK in Android Studio**

1. Download NDK via SDK Manager.
2. In build.gradle (module level), add:

```
groovy
android {
   defaultConfig {
      externalNativeBuild {
         cmake {
            cppFlags "-std=c++11"
         }
      }
   }
   externalNativeBuild {
      cmake {
         path "src/main/cpp/CMakeLists.txt"
      }
   }
}
```

3. Create CMakeLists.txt to describe how to build your native code.

**Real example:** The popular game **PUBG Mobile** uses NDK extensively for its 3D rendering engine (Unreal Engine is written in C++). The Java part only handles Android lifecycle and touch events; the heavy lifting is done by native code.

**Appendices**

**Appendix A: Recommended Textbooks and Online Resources**

**Textbooks (as per course):**

1. Mark L. Murphy, *Beginning Android 2*, Apress.
2. Zigurd Mednieks et al., *Programming Android, 2nd Edition*, O'Reilly.
3. Syed Hashmi et al., *Pro Android 2*, Apress.

**Official Online References:**

- Android Developer Documentation
- Services Guide
- Intents and Intent Filters
- Data Storage
- Graphics
- Sensors

**Appendix B: Sample 20-Mark Exam Question and Answer Outline**

**Question:** Explain the Android Activity Lifecycle in detail. Why is it important to understand it? Provide real-world scenarios and code snippets.

**Answer Outline (to be written in full as per this book):**

1. Definition of Activity and its role.
2. The 7 lifecycle methods – what each does, when called, and what to implement.
3. Diagrams (described or drawn).
4. Scenarios: normal launch, pressing back, rotating screen, incoming call, low memory kill.
5. Saving and restoring instance state with onSaveInstanceState and onRestoreInstanceState.
6. Code example demonstrating all lifecycle logs.
7. Common mistakes (e.g., doing heavy work in onPause, forgetting to call super).
8. Importance: avoiding memory leaks, preserving user data, handling configuration changes.

**End of Book**

This document covers **every single topic** from your course outline (EI-333 Mobile Application Development) in **comprehensive depth**, with **real-life examples**, **code snippets**, **tables**, and **bullet points** where helpful. It is structured like a professional textbook and is suitable for BS IT 6th semester students.

You can now use this as your complete reference for both learning and exam preparation.